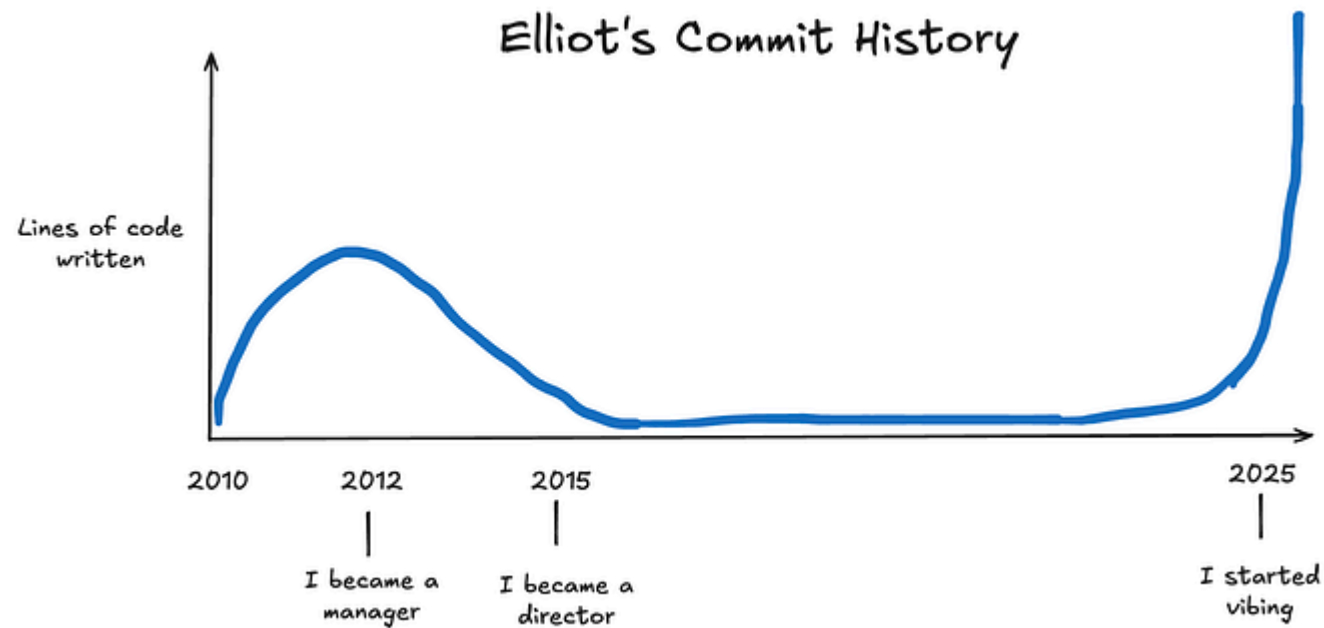


[< Go to the original](#)

Agentic AI has changed my career

I don't write code... or at least, I didn't



Elliot Graebert

Follow

a11y-light · September 11, 2025 (Updated: September 11, 2025) · Free: Yes

I became a manager after only a couple years of experience, and a director shortly after that. Running a team of 20 people meant that I spent all my time managing, and none of my time coding.

I've spent my whole career wishing that I could just "jump into the code" and help the team, but every time I opened up the IDE, the effort vs impact ratio was way off. I was better off managing the project.

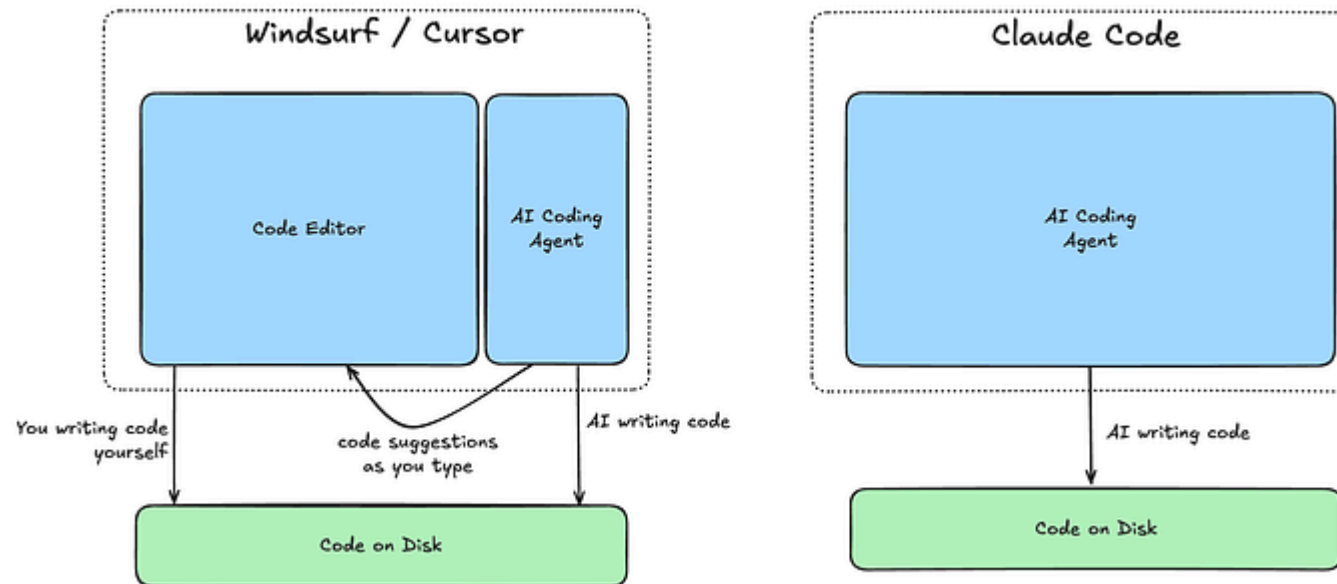
About 4 months ago, I began vibe-coding at full speed, on our main codebase. Not side projects, and not throwaway internal applications. I began vibe coding in a multi-million-line monorepo, on the actual code that controls our drone fleet.

The first 2 months were spent vibe coding, and the second 2 months have been spent on agentic AI. In terms of code submissions, I am now in the top 5 contributors at Skydio, submitting as many as 10 pull requests per hour. My code changes aren't complex, but they are valuable!

Managers and directors: you can contribute to your team's code FAR more than you think.

set this up at your company.

What is Vibe Coding vs Agentic AI?



This is a better question for Google, but here's the quick answer.

AI-Assisted Coding / Vibe Coding is pair programming

programming flow. As the driver, you are writing the code and the AI is giving you helpful suggestions. As the observer, the AI is the one making the changes, and you are just code reviewing it, one line at a time.

Agentic AI is like having an intern

In the agentic model, you are tasking out work to an overly-ambitious intern. The more context you provide the agent, the more likely it will complete the task the way you want. Much like an intern.

The primary distinction between these two is that:

Vibe-coding is an active process with iteration measured in seconds. Agentic AI is a passive process with iteration measured in minutes.

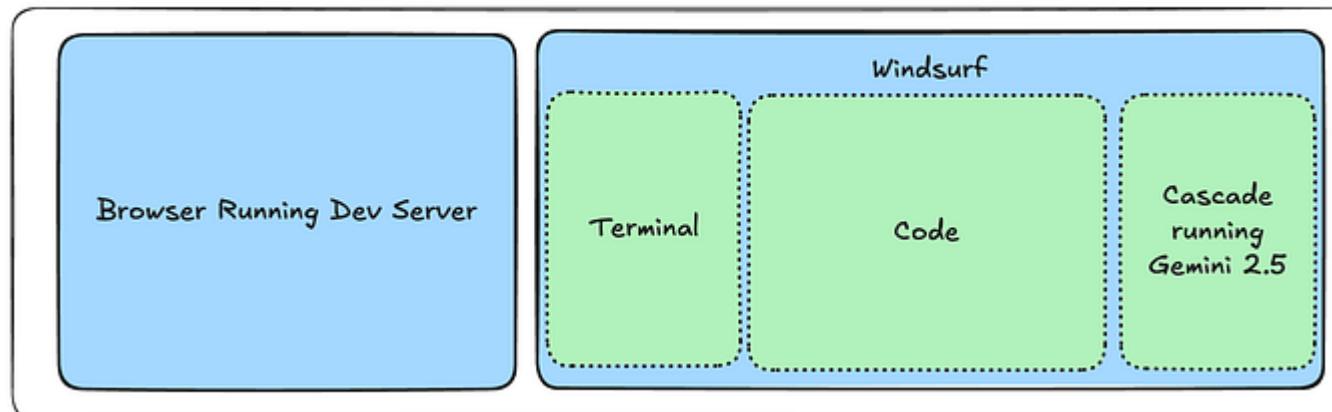
In the next section of the blog post, I'm going to go through my process of ramping up from vibe coding at 0.5x speed to Agentic AI at 10x speed.

Vibe coding at 0.5x speed: just a dude with an editor.

One area where the team said I could be a lot of help was on small design nits and UI bugs that don't require the skills of a full-time engineer, but still eat away at the hours. Things like "change the copy on this page" or "this page looks weird if you shrink the page".

At the time I weighed my options: could I speed my team more by doing more active coaching (i.e., micromanaging), cleaning up the Jira queue (i.e., management self-gratification), or just fixing the bugs myself? Let's try bug squashing instead. Note that I do not (and still don't) know TypeScript.

Elliot's Ultra Wide Monitor



- Then I'd ask Windsurf (Cascade) to make a change.
- Then I'd review it in the browser and iterate if necessary.

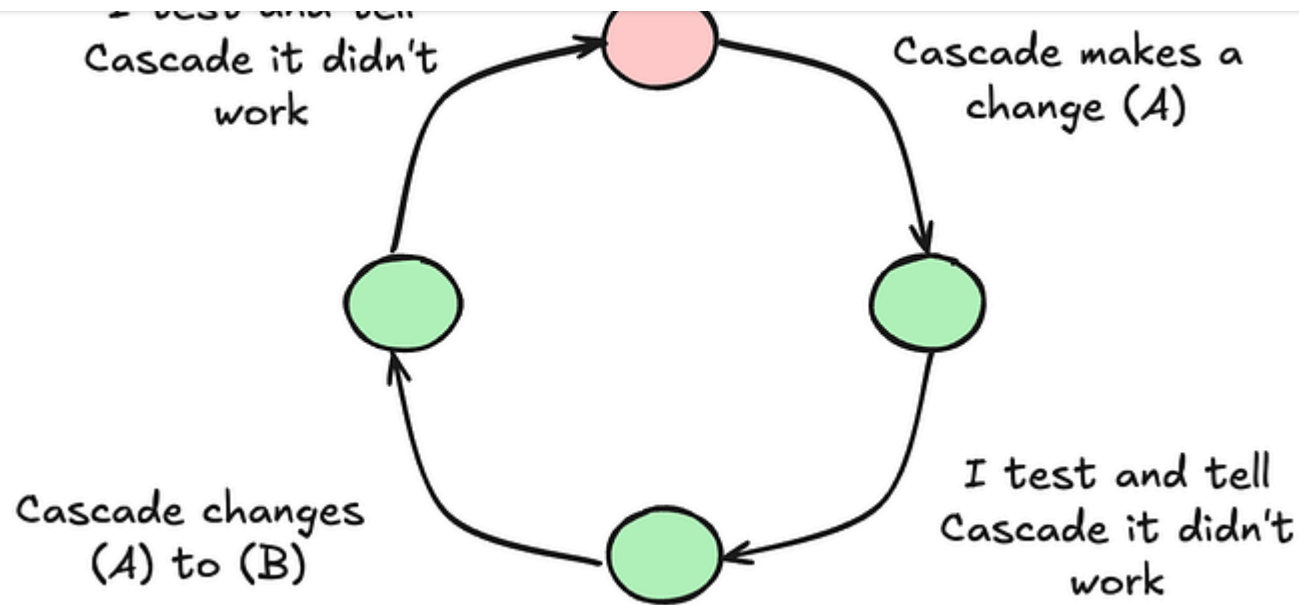
Turns out, our backlog had many small UX bugs that no one was able to find time for. The AI would take multiple rounds to get the change right and the code often needed changes after review, but still, I was closing out ticket after ticket.

Was I faster than a developer? No.

Vibing at 1.5x speed: brute forcing a problem

My first breakthrough: why have 1 machine when you can have 3?

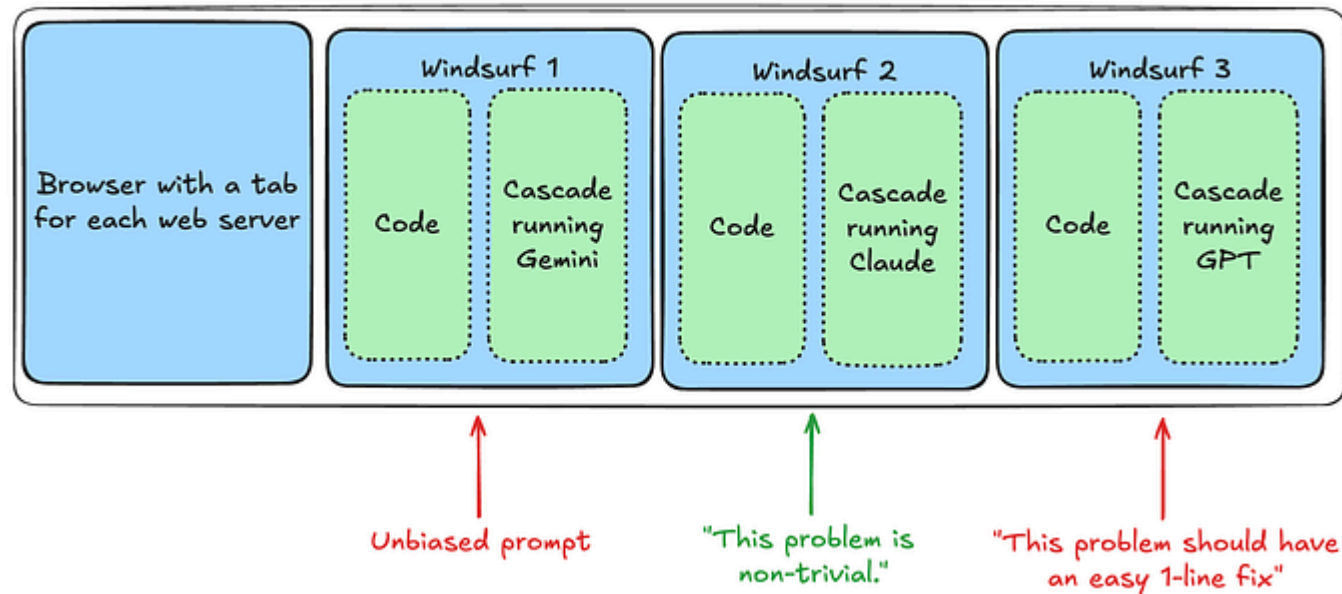
At one point, I got stuck on a bug that seemed so simple at first glance, but was a gnarly issue in the depths of Relay. UGH. I couldn't even tell you the difference between Redux and Relay, much less figure this out. AI got stuck in a bad loop:



From A -> B -> A -> B -> A -> B -> A -> B for over an hour.

I am not one to admit defeat. I will fix this bug. After the conversation had gone on for an hour, the AI was hopelessly stuck. Its context was so deep that it refused to try novel approaches. So I tried something crazy: what if I started:

- 3 new Coder workspaces (remote development machines)
- with 3 different models (Gemini, Claude, and GPT)
- with 3 different prompts



I would test and give feedback to each workspace in turn, hoping that one of them would fix the bug. I basically DDoS'd the bug with optimism. I stopped caring about the code. All that mattered was fixing the bug. I figured that if it fixed the bug, I could clean up the code later. After an hour, the second workspace figured out the solution, and presto: I submitted and merged the code!

This started a new development wave for me: if this works for 3 AI agents on 1 hard problem, I should be able to apply 3 AI agents to 3 easy problems. Now I started running through bugs, tripling my pace.

Vibe Coding at 3x speed with 3x workspaces

Now I'm crushing out tickets. I asked our head of design to write me a list of every piece of UI jank he comes across, and I'd crush it out faster than he could write it down. I'd keep 3 Windsurfs open at a time, testing, giving feedback, and moving on. If a particular Windsurf got stuck, I'd double/triple down by giving the same issue to the next Windsurf. The next biggest challenge was reducing the time spent on the change/test cycle. I needed the AI to make the correct change faster.

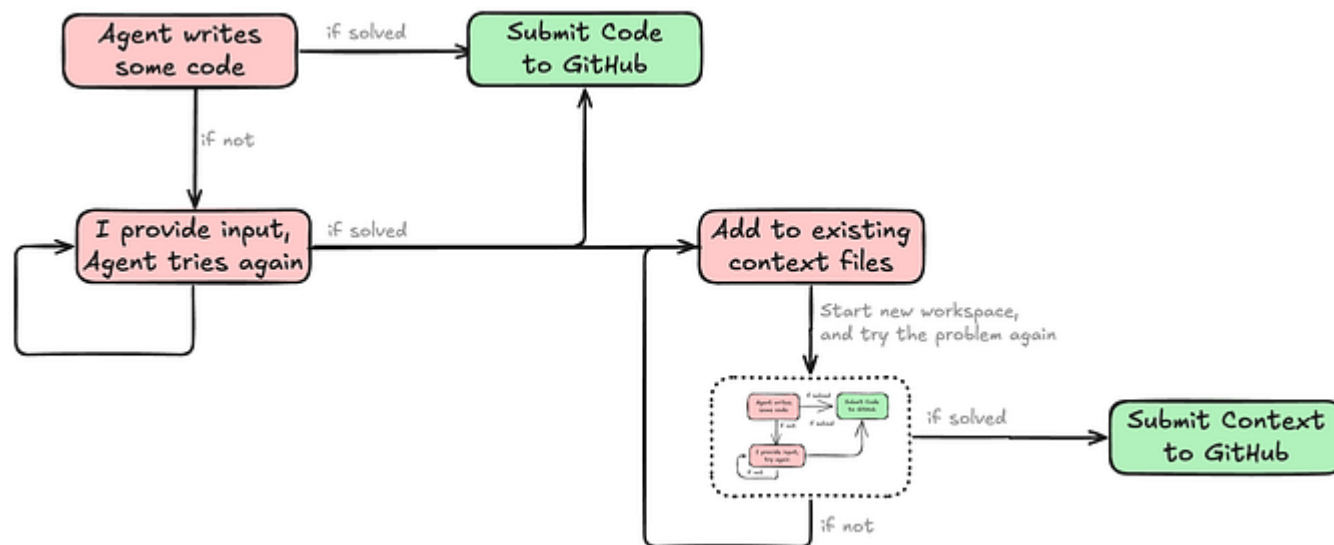
Introducing: Context files (e.g. Windsurf Rules)

Skydio works out of a single large monorepo that is less a codebase and more an archaeological dig over the last 10 years of Skydio history. Half-completed migrations, ancient code, and interns littered the repo with anti-patterns and misinformation. When the AI only had my input and other code as context, the code it wrote could pattern match off of an intern's code from eons ago. Boo.

To get a better result, I needed to feed the AI our coding best practices and anti-patterns to avoid. I needed to tell it which commands it could use to run the unit tests before it pushed to CI. For me, this was Windsurf Rules, which is just a set of

You can imagine rules as just pre-saved prompts that get auto-fed into the system. For example, if your prompt was to edit TypeScript for your frontend app, you could have a corresponding `.windurf/rules/typescript-best-practices.md`. The content in this best-practices guide is essentially auto-injected after your prompt.

Here's how I approached writing this file:



Iterating over-and-over on context is what makes it effective

unlocked the final solution. Then I would make an edit to the context file, start a new workspace (clean slate), and give it the same prompt. If the AI solved it easily on this run, then my context file changes had the intended effect.

💡 **Key insight for managers: AI doesn't automatically solve your problems. You gotta invest in the tooling.**

At this point, I was closing out enough tickets that other developers noticed what was happening, and they started asking more questions about AI workflows.

Was I faster than a developer? Faster than a senior engineer at straightforward work.

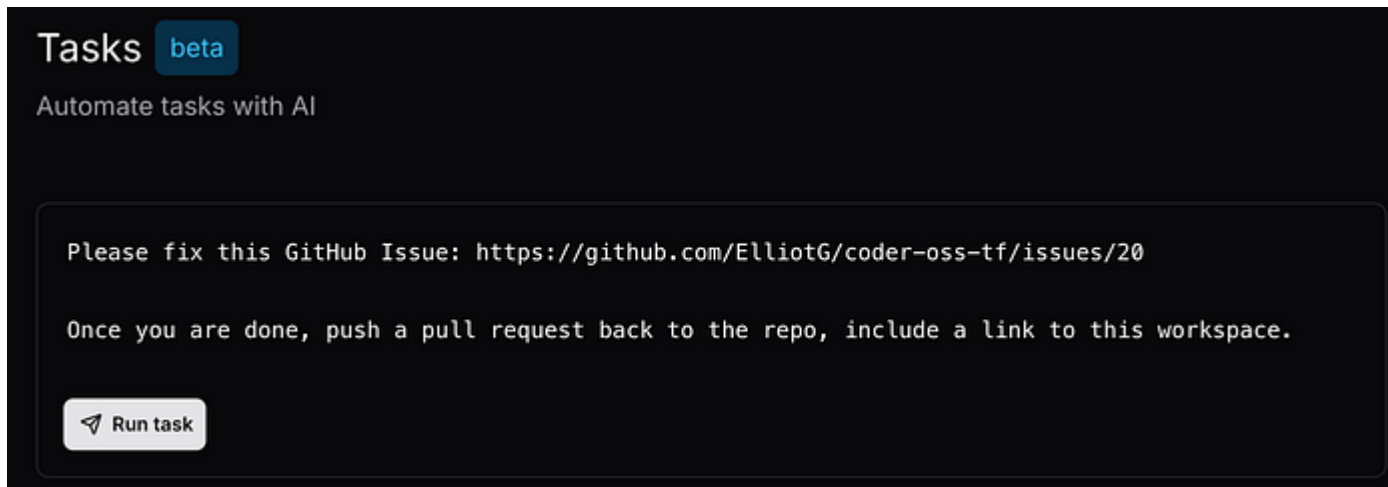
Vibe Coding at 10x speed with 10x workspaces

To triple my productivity again, I needed to take the slowest thing out of the equation: me. I was able to keep about 3 Windsurfs going at a time, but it took all of my attention. If I asked it to perform a long task, I had to keep my Windsurf open and confirm next steps every couple of minutes. I needed to automate the following:

- Setting up the workspace and dev server automatically.

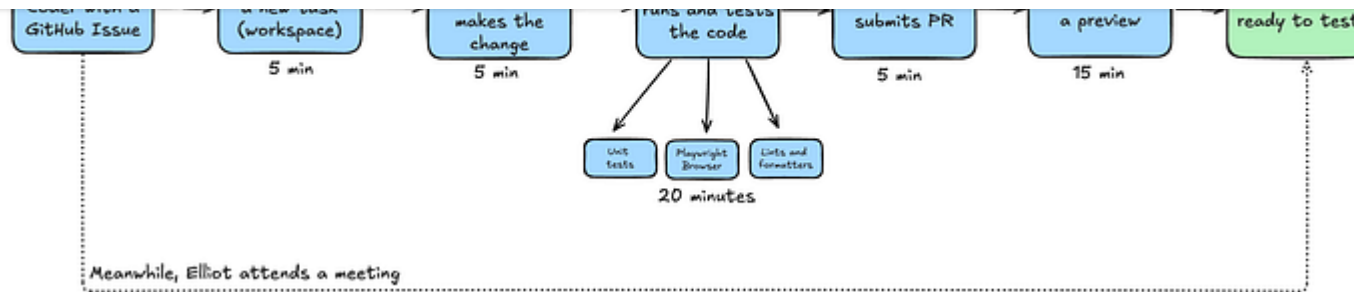
- Drafting the commit and PR in GitHub.

Introducing Coder Tasks



Coder Tasks are a new entrypoint for Agentic AI

Coder Tasks are automated workspaces that are focused around automating the flow from "prompt" → AI agent executing the task. If you set it up to push the code automatically to GitHub, then you can parallelize many tasks and not re-engage until the pull request is ready for review.



In this model, I don't waste time waiting between steps.

It doesn't really matter if the task several hours, as I'm not actively engaged until the AI has completed its change. Also, note that writing a prompt and testing the preview doesn't require technical skill, meaning that:

Agentic AI enables everyone in every role to contribute to making your product better.

Designers can submit color and formatting fixes. Product managers can fix copy. And Skydio's flight test team can submit fixes alongside the bugs they find.

MCPs are the final piece of the puzzle

MCPs are just a fancy way of saying "AI Integration", where "MCP" is the technical standard that Anthropic convinced the world to follow. It allows GitHub (the

Pull Request interactions) and Playwright (Browser interactions)

Introducing MCPS

Gathering Context

- Get input from the user
- Read the code
- Read input from the GitHub Issue
- Reproduce the bug in a headless browser

Validating the Change

- Run the compiler
- Run automated tests
- Verify the bug in a headless browser

Submitting the Change

- Push the code to GitHub
- Craft a PR that includes helpful tips

With the GitHub MCP, you unlock the ability to read from GitHub Issues. This allows you to simplify your prompt to: "Fix this <issue>, while keeping <additional context> in mind." The more important part is that it can create the Pull Request automatically. At Skydio, this allows both long CI checks to run, and it triggers our Vercel preview. This means that I can wait until the Vercel preview is up before I get engaged again. My flow becomes:

1. Kick off several Coder tasks
2. Wait until the Pull Request is pushed, and has a green Vercel build.
3. Test the Vercel preview
4. Trigger the task to read and address my feedback via the Pull Request

With this flow, I am more limited by my ability to think of the next task than I am by the tasks themselves! However, the next step-function improvement is to improve the ability of the AI to iterate without me...

With the Playwright MCP, the AI can navigate around any website, including your development stack! This means that the AI can be told to do the following:

1. Start the development server on fresh code and reproduce the original bug.
2. Make a change, and then test it.

4. If it's not, try again.

Stop and think about how powerful this is. When the feedback cycle was dependent on my response, I could only keep 3 Windsurfs going at a time without feeling overwhelmed. However, when AI can iterate with itself, the limit becomes much higher. I'm able to generate dozens of concurrent tasks, provided I can tell the AI how to verify its change.

The key to successful agentic AI is creating a feedback cycle that doesn't involve a human's response.

This is where the floodgates were unlocked for Skydio. The activation energy required to give a prompt to a Coder Task is near zero. I began running 10 tasks at a time. Here's a (true) example:

- A senior developer drafted up a new component to be our standard for filtering data in tables. It required both a new GraphQL query and replacing a component in the frontend.
- They migrated 2 filters to use as an example.
- I then tasked out over a dozen Coder tasks to finish the entire migration across our app in a single day.

Claude Code is now the number-two developer at Skydio in terms of commit velocity! The sky is the limit!

Next focus: getting AI to perform advanced workflows!

Wrap-up

For the very few of you that made it to the end of this, I appreciate you reading this! This was a journey that I am proud of. Using Agentic AI to bulk-produce pull requests is something that I've been pitching Skydio since the beginning of the year, but there were many building blocks along the way that had to be laid first.

The transformation at Skydio has been amazing, and my team's backlog has never been cleaner. But what has been most exciting is seeing non-developers contribute directly to the product.

Thanks for reading my story!

[#agentic-ai](#) [#ai](#) [#coding](#) [#developer-tools](#)